



中南大學
CENTRAL SOUTH UNIVERSITY

IT 项目系统实践实习报告

姓 名： 杨梓枫

班 级： 计科 2207 班

学 号： 8208220610

学 院： 计算机学院

本科生院制

2025 年 9 月

目录

第 1 章	实习目的	3
1.1	深化专业知识应用	3
1.2	强化 Golang 技术栈掌握	3
1.3	熟悉国际电商业务逻辑	4
1.4	提升团队协作与职业素养	4
1.5	积累项目经验与明确职业方向	5
第 2 章	实习内容和过程	6
2.1	实习公司及部门简介	6
2.1.1	公司概况	6
2.1.2	部门及团队介绍	7
2.2	岗前准备阶段（2025 年 5 月 23 日 —2025 年 5 月 30 日）	8
2.2.1	公司及部门入职培训（5 月 23 日）	8
2.2.2	项目架构与技术栈学习（5 月 24 日 —5 月 26 日）	9
2.2.3	需求评审与业务逻辑理解（5 月 27 日）	12
2.2.4	接口开发实践与测试（5 月 28 日 —5 月 29 日）	13
2.2.5	微服务通信机制学习（5 月 30 日）	14
2.3	核心业务模块开发阶段（2025 年 6 月 2 日 —2025 年 8 月 22 日） ..	14
2.3.1	订单相关接口开发（6 月 2 日 —6 月 13 日）	14
2.3.2	订单履约存储库迁移项目（6 月 14 日 —7 月 15 日）	16
2.3.3	国际电商服务 AI Agent 开发框架搭建（7 月 16 日 —8 月 5 日） ...	19
2.3.4	包裹揽收前轨迹功能开发（8 月 6 日 —8 月 22 日）	21
第 3 章	实习照片	25
3.1	实习证明	31
第 4 章	实习收获与总结	32
4.1	实习收获	32
4.1.1	技术能力提升	32
4.1.2	业务理解加深	34
4.1.3	团队协作能力提升	35
4.1.4	职业素养养成	35
4.2	问题与反思	36
4.3	未来展望	37

第1章 实习目的

毕业实习作为高等教育教学的关键实践环节，是理论知识与企业实际开发场景深度融合的重要桥梁。本次实习严格遵循中南大学计算机学院计科专业毕业实习大纲，紧密结合后端开发（Golang）岗位需求，旨在达成以下核心目标：

1.1 深化专业知识应用

将大学阶段所学的计算机科学与技术理论，包括数据结构、数据库原理、计算机网络等，与企业实际开发场景相结合。通过参与珠海聚横易行网络科技有限公司国际电商部门的后端开发工作，验证并深化对专业知识的理解，提升在实际项目中的应用能力。例如，在订单系统开发中，运用数据库原理优化查询索引，提升数据检索效率。在处理海量订单数据时，原本的查询语句执行效率较低，通过深入分析数据库表结构和查询需求，为常用查询字段添加合适的索引，使得查询时间大幅缩短，从原本的数秒减少到了几百毫秒，极大地提升了系统的响应速度和用户体验。同时，在实现订单状态变更逻辑时，利用数据结构中的状态机概念，确保订单状态的流转符合业务规则，避免出现异常状态，保证了数据的一致性和准确性。在计算机网络方面，通过优化网络传输协议和数据格式，减少了数据传输的延迟和带宽占用，提升了系统的整体性能。

1.2 强化 Golang 技术栈掌握

熟练掌握 Golang 编程语言在后端开发中的核心应用，深入理解其并发编程模型（如 Goroutine 与 Channel）在高并发场景下的优势。熟悉微服务架构设计理念，掌握 gRPC 接口开发规范及服务间通信机制，了解分布式系统中缓存技术（Redis）、消息队列（Kafka）等中间件的实际应用流程，提升复杂技术栈的实践能力。在实际开发中，利用 Golang 的 Goroutine 实现了高效的并发任务处理。比如在订单导出功能中，传统的单线程处理方式在面对大量订单数据时耗时较长，通过使用 Goroutine 并发处理数据读取和 Excel 文件生成的任务，将导

出时间从几分钟缩短到了几十秒，大大提高了工作效率。同时，深入学习 Channel 的使用，实现了 Goroutine 之间安全、高效的通信和数据共享，避免了并发编程中常见的数据竞争问题。在微服务架构方面，参与设计和开发了多个微服务模块，通过 gRPC 实现了服务间的高性能通信，确保了系统的可扩展性和灵活性。对于 Redis 缓存技术，掌握了如何根据业务场景合理设置缓存策略，如针对热门商品详情页面，设置较长的缓存过期时间，减少数据库查询压力，同时通过缓存更新机制保证数据的一致性。在消息队列 Kafka 的应用中，实现了订单状态变更后的异步通知机制，解耦了系统模块，提高了系统的稳定性和可靠性。

1.3 熟悉国际电商业务逻辑

深入了解字节国际电商业务的核心逻辑与流程，包括订单管理、商品库存、支付履约、物流配送等关键模块的业务关联。熟悉电商后端系统的设计规范、开发流程、测试标准及部署流程，掌握如何运用技术手段解决业务痛点，如通过分布式锁确保库存扣减的一致性，培养对复杂业务场景的分析与解决问题能力。在订单管理模块，全面掌握了订单从创建到完成的全生命周期管理，包括订单的创建、支付、履约、退款、统计等环节。在商品库存管理中，深刻理解了库存扣减的业务逻辑和重要性，通过分布式锁技术解决了高并发场景下库存超卖的问题。在支付履约方面，对接第三方支付平台，实现了安全、可靠的支付流程，并处理了支付结果回调、异常处理等复杂业务逻辑。在物流配送模块，与第三方物流 API 对接，实现了物流信息的实时查询和跟踪，为用户提供了良好的购物体验。同时，参与电商后端系统的设计和开发过程，熟悉了从需求分析、架构设计、编码实现到测试部署的全流程，能够根据业务需求和性能指标进行系统优化和问题排查。

1.4 提升团队协作与职业素养

适应企业团队协作模式，积极参与项目需求评审、代码评审、技术分享等活动，学习敏捷开发流程（如两周迭代周期、每日站会机制）。提升沟通协调能

力，与产品经理、前端开发、测试人员等高效协作，确保项目需求准确理解与

实现。培养严谨的职业态度，注重代码质量与文档规范，提升团队合作能力及职业素养。在需求评审中，与产品经理密切沟通，深入理解业务需求，提出技术实现方案和建议，确保需求的可行性和技术合理性。在代码评审中，认真审查代码质量，遵循团队的编码规范，提出改进意见，提高代码的可读性、可维护性和可扩展性。通过每日站会，及时沟通项目进展、问题和风险，确保团队成员之间信息共享，协同工作。在技术分享活动中，不仅分享自己的技术经验和学习成果，还学习了团队成员在不同技术领域的知识和实践经验，拓宽了技术视野。同时，注重与前端开发人员和测试人员的协作，确保前后端数据交互的准确性和稳定性，通过有效的沟通和协作，及时解决测试过程中发现的问题，提高项目的质量和交付效率。

1.5 积累项目经验与明确职业方向

通过参与真实项目的开发与维护，积累企业级项目经验，熟悉从需求分析到上线部署的全流程。在实践中发现自身专业能力短板，如分布式事务处理、微服务治理等领域的不足，为后续职业发展制定针对性提升计划，明确后端开发的职业方向。在整个实习过程中，参与了多个核心业务模块的开发和优化工作，从最初的需求分析和设计，到编码实现、测试、上线以及后续的维护，全程跟进项目的进展。在这个过程中，遇到了各种各样的技术问题和挑战，如分布式系统中的数据一致性问题、高并发场景下的性能瓶颈等，通过查阅资料、请教同事和深入研究，成功解决了这些问题，积累了宝贵的项目经验。同时，也意识到自己在分布式事务处理和微服务治理等方面的知识和经验不足，明确了后续需要进一步学习和提升的方向。基于本次实习的经历和收获，坚定了从事后端开发的职业方向，制定了长期的职业发展规划，计划在未来深入学习分布式系统、微服务架构、高并发编程等领域的知识，不断提升自己的技术水平和业务能力，努力成为一名优秀的后端开发工程师。

第2章 实习内容和过程

2.1 实习公司及部门简介

2.1.1 公司概况

珠海聚横易行网络科技有限公司自 2018 年成立以来，便在跨境电商技术服务领域崭露头角。公司依托字节跳动强大的生态优势，以技术创新为驱动，深度融入国际电商市场。在短短几年内，公司从一个初出茅庐的创业公司，发展成为行业内的佼佼者，为全球 20 多个国家和地区的商家提供了全方位的跨境电商解决方案。其服务范围涵盖店铺运营、供应链管理、物流配送以及技术支持等多个关键环节，为跨境商家提供了一站式的服务，帮助他们在国际电商市场中取得成功。

部门现有员工 1000 余人，其中产研技术团队占比超过 80%，这些技术人才成为公司创新发展的核心力量。核心研发人员来自国内外知名互联网企业，他们不仅具备深厚的技术功底，还拥有丰富的跨境电商技术研发经验。在过去的项目中，他们成功解决了诸多技术难题，例如在高并发场景下，通过优化系统架构和算法，实现了系统的稳定运行，确保了商家业务的正常开展。公司秉持“技术驱动电商创新”的理念，将技术创新视为公司发展的核心动力。在技术研发上持续投入，不断探索新技术在跨境电商领域的应用，先后推出多款跨境电商管理系统。这些系统凭借其高效的性能、稳定的运行和丰富的功能，赢得了市场的广泛认可，与超过 50000 家跨境商家建立了长期稳定的合作关系。

凭借优质的技术服务和创新的产品解决方案，公司连续 3 年被评为“跨境电商技术服务领军企业”。这一荣誉不仅是对公司技术实力和服务质量的高度认可，也是公司在行业内影响力不断提升的体现。同时，公司还获得了多项软件著作权和专利证书，这些知识产权是公司技术创新的结晶，为公司的持续发展提供了坚实的技术保障。

表 1 公司优势说明

公司核心优势	具体说明
技术优势	拥有成熟的微服务架构体系，具备高并发、高可用、可扩展的系统设计能力，核心技术团队具备 10 年以上电商技术研发经验
生态优势	深度对接字节跳动海外流量生态，为商家提供精准的流量扶持和营销推广方案
服务优势	提供 7×24 小时技术支持服务，建立快速响应机制，保障商家业务顺畅运行
资源优势	与全球多家物流企业、支付机构建立战略合作，为商家提供高效、安全的物流和支付解决方案

2.1.2 部门及团队介绍

字节国际电商部门作为公司的核心业务部门，肩负着字节跳动海外电商平台后端系统研发与维护的重任。该部门下设计划中心、商品中心、用户中心、支付中心、物流中心等 5 个核心研发团队，这些团队犹如紧密协作的齿轮，共同推动着电商平台的稳定运行。部门共有研发人员 885 人，其中后端开发人员 400 人，他们负责编写高效、稳定的后端代码，实现业务逻辑和数据处理；前端开发人员 257 人，致力于打造用户友好、界面美观的前端界面，提升用户体验；测试人员 65 人，通过严谨的测试流程，确保系统的质量和稳定性；产品经理 110 人，负责把握产品方向，协调各方资源，推动产品的迭代和优化。

本人所在的订单中心研发团队，专注于国际电商订单全生命周期的后端系统开发。从订单的创建，到支付、履约、退款以及统计等各个环节，都离不开团队的精心设计和开发。团队采用敏捷开发模式，以两周为一个迭代周期，通过每日站会、迭代规划会、迭代评审会等机制，确保项目的高效推进。在每日站会中，团队成员可以及时沟通项目进展、遇到的问题以及解决方案，保证信

息的及时共享和问题的快速解决；迭代规划会则用于明确下一个迭代周期的任务和目标，合理分配资源；迭代评审会对迭代成果进行评估和审核，确保产品质量。

团队的技术栈以 Golang 为主，结合 MySQL、Redis、Kafka、Elasticsearch 等中间件，构建了高并发、高可靠的订单系统，能够支撑每日百万级订单的处理需求。在处理高并发订单时，Golang 的并发编程特性和中间件的协同工作，使得系统能够快速响应，保证订单处理的高效性和准确性。例如，在促销活动期间，订单量会瞬间激增，团队通过优化 Golang 代码和合理配置中间件，成功应对了高并发挑战，确保了系统的稳定运行。

2.2 岗前准备阶段（2025 年 5 月 23 日 —2025 年 5 月 30 日）

2.2.1 公司及部门入职培训（5 月 23 日）

2025 年 5 月 23 日清晨，我抱着印着字节 logo 的入职礼包，提前十分钟走进了培训会议室——长桌旁已经坐了不少同批入职的伙伴，屏幕上正循环播放着公司海外电商业务的宣传短片，轻快的背景音乐里，能听到大家小声交流的期待。培训正式开始后，人力资源部门的讲师先打开了一份泛黄的老照片 PPT：第一张是公司初创时十几个人挤在小办公室的场景，白板上写满了跨境电商系统的早期架构草稿；接着是团队第一次拿下十万级日订单时的庆祝合照，每个人脸上都沾着蛋糕奶油；最后定格在现在覆盖全球 150+ 国家的业务版图——从最初啃下东南亚市场的“冷启动”难关，到突破欧美合规壁垒站稳脚跟，再到如今成为跨境电商技术服务领域的头部玩家，每一段历程的细节都让我真切感受到，这家公司的“奋斗”不是口号，是写在每一个项目里程碑里的实在足迹。

讲到企业文化时，讲师举了个具体案例：去年大促前，技术团队为了优化订单系统的并发性能，连续一周泡在机房，最终把支付响应延迟从 800ms 压到了 150ms，而驱动他们加班的，正是“技术驱动电商创新”这一核心价值观——不是为了完成 KPI，是真的想让海外商家能在大促里接住每一笔订单，让用户能顺畅完成支付。跟着这部分内容，我还逐条学习了公司的规章制度：从代码

提交的权限分级，到跨境数据的保密红线，甚至连办公区的设备借用流程都讲得很细致，尤其是保密协议里“禁止私自留存业务数据”的条款，讲师特意强调了两个因违规导致项目风险的案例，让我瞬间绷紧了“守规矩、明边界”的弦，清楚了自己作为技术人员在数据安全和合规上要扛的责任。

上午的公司培训刚结束，部门的资深业务专家就抱着一沓流程手册走进了会议室，下午的专项培训直接从“实战案例”切入：他先打开了一个商家入驻的真实后台界面，指着“资质审核”模块说，这里不仅要校验营业执照的真实性，还要对接海外本地的工商系统做交叉验证，曾经有个商家用 PS 的执照提交，就是被这个交叉验证环节拦了下来；说到商品上架，他又点开了一张图片处理的对比图——左边是商家原上传的模糊主图，右边是系统自动优化后的高清图，背后是部门自研的图像压缩算法，既能保证画质又能加快加载速度。从用户下单时的“订单生成 - 库存预扣 - 支付回调”三连流程，到支付履约时“资金清分要精准到每个币种的汇率小数点后四位”，再到物流配送里“订单分配要优先匹配最近的本地仓以缩短时效”，每一个环节他都配上了流程图和实际问题案例，比如曾经因为物流跟踪接口的延迟，导致某批订单的用户投诉率涨了 12%，后来团队优化了接口的异步回调机制才解决。

培训最后半小时是业务知识测试，20 道题里既有“商家入驻需要哪三类资质”这样的基础题，也有“订单支付失败后库存预扣如何回滚”这样的逻辑题，我凭着下午记的满满三页笔记，很快就答完了所有题目，交卷时专家扫了一眼我的答案，笑着说“这几道逻辑题答得挺准，看来是真听进去了”。等成绩出来，我拿了 96 分，扣的 4 分是漏答了一个资质审核的辅助材料项——这也让我记牢了“跨境业务里每一个细节都不能漏”的道理，算是给接下来的技术开发提前打了个“严谨”的预防针。

2.2.2 项目架构与技术栈学习（5 月 24 日 — 5 月 26 日）

2025 年 5 月 24 日一早，我便带着提前准备好的笔记本和电脑，跟着指导老师进入了技术研讨室，正式开启了字节国际电商后端项目架构的深度学习之旅，这场学习一直持续到 5 月 26 日，三天的时间里，每一分每一秒都充满了知

识的沉淀与思维的碰撞。指导老师并没有一上来就抛出复杂的理论，而是先拿出了一张超大尺寸的项目架构总览图，铺在会议桌上，用不同颜色的马克笔标注出各个核心模块。他指着图上交错的线条和标注的服务名称，耐心地为拆解微服务架构的设计理念：“我们做跨境电商后端系统，面对的是全球不同区域的用户、海量的订单数据和复杂的业务场景，要是用传统的单体架构，一旦某个模块出问题，整个系统都可能瘫痪，而且后续迭代和扩展也会特别麻烦。”他解释道，微服务架构的核心就是“拆分”与“独立”，把原本庞大的电商系统，按照业务域拆分成订单中心、商品中心、用户中心、支付中心、物流中心等一个个小型的、独立的服务，每个服务都有自己专属的开发团队、数据库和部署流程，不仅能实现并行开发，提高迭代效率，还能根据不同服务的流量需求单独扩容，比如大促期间订单量激增，就只针对订单中心服务扩容，不用浪费资源对整个系统进行调整，这样一来，系统的可维护性、可扩展性和灵活性都得到了极大提升。

为了让我更直观地理解，指导老师还打开了公司的代码仓库，调出了订单中心和商品中心的实际代码示例。他从代码的入口函数开始，一步步带我梳理服务的启动流程、核心业务逻辑的实现的代码块，重点讲解了两个中心的职责划分：订单中心主要负责订单的创建、状态变更、履约跟踪、退款处理等全生命周期管理，是整个电商交易链路的核心枢纽；商品中心则承担着商品信息的录入、审核、分类、库存管理等功能，所有前端展示的商品数据、下单时的库存校验，都依赖于商品中心的服务支撑。在讲解过程中，我还了解到不同微服务之间的接口定义规范和通信方式——项目中主要采用 **gRPC** 作为跨服务通信的核心协议，通过 **Protobuf** 定义标准化的接口和数据结构，确保不同服务之间的数据交互准确、高效。指导老师特意用一个“用户下单后库存扣减”的场景举例：用户下单时，订单中心会通过 **gRPC** 接口调用商品中心的库存查询服务，确认库存充足后，再发送库存扣减请求，商品中心完成扣减后返回成功响应，订单中心才会继续推进后续的支付流程，整个过程通过同步调用保证数据一致性，而对于一些非核心的异步通知场景，则会通过 **Kafka** 消息队列实现解耦，避免核心流程阻塞。

在架构学习的同时，技术栈的系统学习也同步展开，其中 **Golang** 的开发规范是重点内容。指导老师专门整理了一份公司内部的《**Golang** 开发手册》，从基础的命名规范开始，逐字逐句地为我讲解：变量名、函数名必须具有清晰的描述性，严格遵循驼峰命名法，比如表示“用户订单列表”的变量不能简单命名为“`orderList`”，而要更精准地命名为“`userOrderList`”，如果是全局变量，则需要以大写字母开头；注释规范更是要求细致，不仅函数头部要包含功能描述、参数说明、返回值含义，对于代码中复杂的业务逻辑、边界条件处理，也要添加行内注释，方便后续其他开发人员理解和维护，指导老师还特意对比了两段代码，一段是注释完善、命名规范的代码，另一段是命名模糊、无任何注释的代码，两者的可读性差异一目了然，让我深刻意识到规范编码的重要性。除了编码规范，目录结构规范也同样关键，项目采用标准的 **Go Module** 目录结构，按照“`cmd`（程序入口）、`internal`（内部代码）、`pkg`（公共包）、`api`（接口定义）、`config`（配置文件）、`scripts`（脚本工具）”等功能模块进行划分，每个微服务都遵循统一的目录标准，这样即使是新人也能快速找到对应的代码位置，极大降低了项目的维护成本。

工具的使用技巧学习则更偏向实践操作。指导老师坐在我旁边，手把手教我使用 **Goland** 开发工具：从项目的克隆与导入、代码的自动格式化与语法检查，到断点调试的设置、变量值的实时查看，再到依赖包的管理与更新，每一个操作步骤都讲解得十分细致。比如调试代码时，他告诉我可以通过条件断点只在特定参数触发时暂停，还可以使用日志断点在不修改代码的情况下输出调试信息，这些技巧能极大提高问题排查的效率。**Git** 版本控制工具的学习也让我收获颇丰，指导老师带我实操了从创建分支、提交代码、解决冲突到合并分支的完整流程，重点强调了公司的分支管理规范：项目主分支为 `main`，开发分支从 `develop` 分支创建，每个功能开发对应一个 `feature` 分支，`bug` 修复则使用 `hotfix` 分支，代码提交前必须先拉取最新代码解决冲突，提交信息要清晰描述修改内容，比如“`fix: 修复订单列表查询时时间范围筛选错误`”，确保代码的安全性和可追溯性。此外，**Jira** 项目管理工具的使用也不可或缺，我学会了如何在 **Jira** 上创建任务、分配负责人、设置截止时间，如何通过看板跟踪任务进度（待处

理、进行中、代码评审、已完成)，以及如何提交问题工单、关联相关需求，这些工具的熟练使用，为后续参与项目协同开发奠定了基础。

为了让我更好地将架构知识和技术栈结合起来，深入理解项目的数据流转逻辑，指导老师安排我协助整理项目的数据库表结构。我首先从订单中心的核心数据表入手，逐一梳理订单表（`order`）、订单明细表（`order_item`）、支付记录表（`payment_record`）、退款记录表（`refund_record`）等关键表的字段信息。在整理过程中，我不仅要记录每个字段的名称、数据类型、长度限制，还要明确字段的业务含义和关联关系。比如订单表中的“`order_status`”字段，数据类型为 `tinyint`，取值范围从 0 到 6，分别对应“待支付、已支付、待发货、已发货、已完成、已取消、退款中”七种状态，每个状态的流转都有严格的业务规则；而订单明细表则通过“`order_id`”字段与订单表建立外键关联，记录每笔订单对应的商品信息、购买数量、单价等明细数据。为了更清晰地展示表间关系，我用 Visio 绘制了数据库表关系图，将核心表之间的关联链路可视化。在整理完成后，指导老师还让我练习使用 Golang 结合 GORM 框架编写简单的数据库查询语句，比如查询特定用户（`user_id=xxx`）在指定时间范围（`create_time between '2025-05-01' and '2025-05-20'`）内的所有订单信息，要求返回订单号、下单时间、订单金额、订单状态等字段；还有统计某一时间段内不同状态的订单数量，生成分组统计结果。在编写过程中，我遇到了时间格式转换、关联查询效率低等问题，指导老师都耐心地为了解答，比如通过使用 GORM 的 `Preload` 方法实现预加载，避免 N+1 查询问题，通过添加索引优化查询效率。通过这一系列的实操练习，我不仅对订单数据的存储和管理逻辑有了更深入的认识，也初步掌握了 Golang 操作数据库的核心技巧，实现了理论知识与实践操作的有效结合。

2.2.3 需求评审与业务逻辑理解（5 月 27 日）

5 月 27 日，我有幸参与了订单列表查询接口开发需求评审会。在会上，产品经理详细阐述了接口的业务需求，包括支持按订单号、用户 ID、时间范围、订单状态等多种条件筛选订单，以满足不同用户的查询需求；支持分页查询，确保在处理大量订单数据时，能够快速响应，提升用户体验；要求响应时间不

超过 300ms，这对接口的性能提出了很高的要求。

通过与团队成员的深入交流和讨论，我不仅深入理解了国际电商订单模块的业务逻辑，还学习了需求评审的流程和方法。在评审过程中，团队成员从不同角度提出问题和建议，如接口的安全性、可扩展性、性能优化等，这让我意识到一个好的需求评审不仅要关注业务需求的实现，还要考虑到系统的整体架构和未来的发展。通过这次评审会，我明确了接口开发的核心需求和技术难点，为后续的开发工作做好了充分准备。

2.2.4 接口开发实践与测试（5 月 28 日 —5 月 29 日）

5 月 28 日至 29 日，我基于 Golang 开始实现订单列表查询接口初稿。在开发过程中，我严格遵循项目开发规范，首先实现了参数接收功能，确保能够准确获取用户传入的查询条件；然后进行参数校验，对用户输入的数据进行合法性检查，如订单号格式是否正确、时间范围是否合理等，防止非法数据对系统造成影响；接着进行数据库查询，根据用户的查询条件从数据库中检索相关订单数据；最后进行结果封装，将查询结果按照规定的格式返回给前端。

完成接口初稿后，我使用 go test 工具编写测试用例，对接口进行全面的测试。测试用例覆盖了正常查询、异常参数、无数据查询等多种场景，以确保接口在各种情况下都能正确运行。在正常查询场景中，输入合法的查询条件，验证接口是否能够返回正确的订单列表；在异常参数场景中，故意输入错误的参数，检查接口是否能够正确处理并返回相应的错误提示；在无数据查询场景中，设置查询条件使得数据库中没有匹配的订单数据，验证接口是否能够正确返回空列表。

在测试过程中，我发现了一些问题，如参数校验不严谨，对于一些边界情况没有进行充分的检查；查询效率低，在处理大量数据时，查询时间较长。针对这些问题，在指导老师的指导下，我进行了优化。对于参数校验问题，我增加了更多的校验规则，对参数的类型、长度、范围等进行严格检查，并完善了错误提示信息，以便于前端开发人员快速定位问题。对于查询效率问题，我通过优化查询逻辑，如调整查询语句的执行顺序、使用索引覆盖等方式，减少了

数据库的查询时间；同时，对数据库进行了索引优化，为常用的查询字段添加合适的索引，进一步提升了查询效率。经过优化后，接口的性能得到了显著提升，稳定性和可靠性也得到了保障。

2.2.5 微服务通信机制学习（5月30日）

5月30日，我系统学习了项目中微服务通信机制，重点了解了 gRPC 的原理与应用。gRPC 是一种高性能、开源的远程过程调用（RPC）框架，它基于 HTTP/2 协议，使用 protobuf 作为接口定义语言，具有高效、灵活、可扩展等优点。在项目中，gRPC 主要用于微服务之间的通信，实现服务间的功能调用和数据传输。

我深入学习了 protobuf 文件的编写规范，了解了如何定义服务接口、请求消息和响应消息的结构。通过定义 protobuf 文件，可以清晰地描述服务的功能 and 数据格式，使得不同微服务之间能够准确地进行通信。同时，我还掌握了 gRPC 接口定义、生成、调用流程，学会了如何使用 protobuf 编译器生成 Golang 代码，以及如何在 Golang 项目中调用 gRPC 接口。

为了更好地理解 gRPC 的应用，我仔细分析了项目现有 gRPC 接口代码，从代码中学习到了服务间通信的实现方式和最佳实践。在分析过程中，我关注了代码中的错误处理、日志记录、性能优化等方面，这些都是保证 gRPC 服务稳定运行的重要因素。最后，我动手练习编写了简单的 gRPC 服务，通过实际操作加深了对 gRPC 的理解和掌握，为参与微服务开发打下了坚实的基础。

2.3 核心业务模块开发阶段（2025年6月2日—2025年8月22日）

2.3.1 订单相关接口开发（6月2日—6月13日）

订单状态变更接口开发（6月2日—6月3日）：协助开发订单状态变更接口，这是一个关键的功能模块，它负责处理订单从待支付、已支付、待发货、已发货、已完成、已取消等多种状态的变更逻辑。在开发过程中，我严格按照业务规则实现状态变更的校验逻辑，例如，只有在订单处于待支付状态时，才能进行支付操作并将订单状态变更为已支付；只有在订单已支付且库存充足的情况下，才能将订单状态变更为待发货等。通过严谨的校验逻辑，确保了订单

状态变更的合法性和一致性，避免了因状态变更错误而导致的业务问题。同时，我还编写了接口文档初稿，详细记录了接口地址、请求参数、响应参数、错误码等信息，为后续的接口调用和维护提供了便利。完成开发后，积极对接前端进行联调，在联调过程中，遇到了跨域及参数格式不匹配等问题。对于跨域问题，通过在后端配置 CORS（跨域资源共享）头信息，允许前端请求跨域访问；对于参数格式不匹配问题，仔细检查了前端和后端的数据格式定义，统一了参数的传递格式，最终确保了接口的正常调用。

日志收集与异常排查（6月4日）：学习项目中使用的日志收集与分析工具（ELK），ELK 是 Elasticsearch、Logstash、Kibana 的缩写，它是一套强大的日志管理和分析解决方案。了解了日志的收集、存储、检索及分析流程，Logstash 负责从各种数据源收集日志数据，对数据进行过滤、转换等处理后，将其发送到 Elasticsearch 中进行存储；Elasticsearch 是一个分布式搜索引擎，用于高效地存储和检索日志数据；Kibana 则提供了一个可视化界面，方便用户对日志数据进行查询、分析和展示。协助排查接口测试中的异常日志，通过分析日志定位问题原因，如数据库连接超时，可能是由于数据库服务器负载过高或网络不稳定导致；参数传递错误，可能是前端传递的参数格式不正确或后端校验逻辑不完善。在定位问题后，配合开发人员进行问题修复，通过优化数据库连接配置、完善参数校验逻辑等方式，解决了异常问题，提升了系统的可观测性和稳定性。

代码评审与整改（6月5日）：参与项目代码评审会，这是一个提升代码质量和团队协作的重要活动。学习了代码评审的标准和方法，评审人员从代码的可读性、可维护性、性能、安全性等多个角度对代码进行审查。针对前期开发的订单列表查询接口和订单状态变更接口代码，评审人员提出了代码冗余，如存在重复的代码片段，可以通过提取公共函数进行优化；命名不规范，变量名和函数名不能准确反映其功能，需要按照命名规范进行修改；错误处理不完整，对于一些异常情况没有进行妥善的处理，可能导致系统崩溃或数据不一致。根据评审建议，对代码进行了全面的整改优化，删除了冗余代码，提高了

代码的简洁性；修改了不规范的命名，使代码更易于理解和维护；完善了错误处理机制，增加了异常捕获和处理逻辑，确保系统在遇到异常时能够稳定运行。通过代码评审和整改，不仅提升了代码质量，还学习了团队其他成员的优秀编程习惯和经验。

订单导出接口开发与优化（6月6日—6月9日）：开发订单导出接口，实现按时间范围筛选数据并导出为 Excel 文件的功能。在开发过程中，使用 Golang 第三方库（如 excelize）处理 Excel 文件的生成，通过该库提供的 API，能够方便地创建 Excel 工作表、写入数据、设置单元格格式等。同时，为了提升导出效率，对数据查询逻辑进行了优化，采用了分页查询的方式，避免一次性查询大量数据导致内存溢出。完成开发后，对订单导出接口进行了并发性能测试，使用压力测试工具（如 JMeter）模拟高并发场景下的订单导出请求。通过压力测试发现数据库查询存在瓶颈，查询时间较长，影响了导出效率。针对这一问题，对数据库查询索引进行了优化，新增了时间范围、用户 ID 字段联合索引，使得查询能够更快地定位到所需数据；同时，采用了分段查询并异步生成 Excel 的方式，将数据分成多个小段进行查询和处理，然后异步生成 Excel 文件，避免了长时间占用主线程，提高了系统的并发处理能力。经过优化后，查询时间从 500ms 缩短至 100ms 以内，并且支持百万级数据导出，无内存溢出问题，大大提升了订单导出接口的性能。

2.3.2 订单履约存储库迁移项目（6月14日—7月15日）

6月14日至7月15日，我深度参与了履约模块核心状态存储从 RDS 到 Redis 的迁移项目。作为解决履约模块长期存在的性能瓶颈的核心攻坚任务，该项目直接关系到海外大促期间订单履约的稳定性与用户体验，也是部门年度技术优化的重点任务之一。项目启动之初，我主动牵头承担了现状调研与技术预研工作，联合测试团队使用 JMeter 压测工具，模拟大促期间每秒万级的峰值业务流量，对原有 RDS 存储的性能表现进行了全面摸底，最终明确了三大核心痛点。

其一，共享数据库域存在严重的业务逻辑交叉问题。订单履约、库存扣

减、物流调度等多个模块共用同一数据库实例，读写请求高峰时段相互抢占资源，导致数据库锁等待时间最长可达 800ms，部分履约状态更新请求因锁冲突出现超时重试；其二，历史订单数据查询频繁引发的性能拖累。商家对账、平台数据分析等场景需频繁查询近 90 天的历史履约数据，这类查询涉及多表关联与复杂过滤条件，慢查询占比高达 35%，平均查询耗时超过 1.2 秒，严重拖慢了整体接口响应速度；其三，RDS 的写性能存在物理上限。随着海外业务扩张，订单量持续攀升，大促期间每秒履约状态变更请求突破 1.5 万次，原有 RDS 的写入吞吐量已接近极限，多次出现数据写入延迟，影响了履约流程的顺畅推进。

针对这些痛点，我投入一周时间深入研读公司自研的最终一致事务组件源码，逐行梳理组件的事务提交流程、异常回滚机制与定时补偿逻辑，结合履约模块的业务特性，制定了“数据映射-增量同步-灰度切换”的分阶段迁移方案。第一阶段为数据映射与结构设计，核心是将 RDS 中结构化的履约状态数据，转换为 Redis 适配的 KV 键值对结构。我参考行业最佳实践，设计了“业务域:模块名:主键 ID”的规范键名格式，例如“fulfillment:order:123456”对应某笔订单的履约状态数据，既保证了可读性，又避免了键值冲突，同时控制键名长度不超过 44 字节，优化内存存储效率。针对不同类型的履约数据，设置了差异化的过期策略：高频访问的在途订单状态数据保留 90 天，已完成履约的历史数据保留 30 天后自动归档，无效状态数据设置 24 小时过期，减少 Redis 内存占用。

第二阶段为增量同步开发，重点解决迁移过程中的数据一致性问题。我采用基于 binlog 日志的同步方案，通过监听 RDS 的 binlog 变更，实时捕获履约状态的新增、修改、删除操作，再通过同步程序将变更数据推送至 Redis，确保两端数据实时对齐。为了验证同步准确性，我专门开发了数据同步校验工具，通过定时对比 RDS 与 Redis 中核心字段的哈希值，自动检测数据差异，一旦发现不一致则触发告警并执行数据订正，有效规避了迁移过程中的数据丢失风险。第三阶段为灰度发布与回滚预案，为保障业务连续性，我们设计了流量梯度切换策略：先将 10% 的非核心业务流量切换至 Redis 存储，持续监控接口响应时

间、数据一致性、Redis 资源占用等关键指标；待 24 小时无异常后，将流量逐步提升至 30%、50%，最终实现全量切换。同时，在系统中预留了一键回滚开关，若灰度过程中出现性能波动或数据问题，可在 1 分钟内将流量切回 RDS，最大限度降低业务影响。

在方案落地阶段，我重点负责 19 个需保持最终一致的核心业务场景初始化逻辑重构。这些场景涵盖订单支付履约、库存扣减履约、物流发货履约、售后退款履约等关键链路，每一个场景都涉及跨服务的数据交互，对一致性要求极高。我逐一拆解原有逻辑中与 RDS 强耦合的事务代码，引入 Redis 的事务与 Lua 脚本能力，将多个关联操作封装为 Lua 脚本执行——借助 Redis 单线程执行模型，确保脚本中的所有命令原子性执行，要么全部成功，要么全部失败，避免了并发场景下的数据不一致问题。针对分布式事务场景，例如订单支付后同步触发库存扣减与履约单创建，我采用“本地消息表+定时任务补偿”的方案：在本地事务中同时完成业务操作与消息记录，再通过定时任务扫描消息表，将未处理的消息异步推送至消息队列，由下游服务消费并执行对应的 Redis 操作，即使中间环节出现异常，也能通过重试机制保证最终一致性。

为了全面验证迁移后的系统稳定性，我搭建了专项测试环境，编写了涵盖正常读写、异常中断、峰值压力、边界条件等多维度的测试用例，累计执行测试用例 200 余次。测试过程中，先后发现并修复了数据同步延迟、缓存击穿、高并发下 Lua 脚本执行超时等问题 12 个。例如，针对热点订单状态查询可能引发的缓存击穿问题，我通过添加互斥锁与缓存预热机制，有效避免了大量请求穿透至 RDS；针对 Lua 脚本执行超时问题，优化了脚本逻辑，将复杂计算拆分为多个轻量脚本，提升执行效率。

经过一个月的开发、测试与优化，迁移项目于 7 月 15 日顺利完成全量上线。上线后的数据监测显示，系统性能得到了质的提升：proxy 端平均延迟从原来的 650ms 降至 10ms，降幅达 98%；Redis 的写性能突破了 RDS 原有上限，大促期间每秒可稳定支撑 2 万次履约状态变更请求，完全满足业务增长需求；慢查询占比从 35% 降至 0.5% 以下，共享数据库域的读写干扰问题被彻底解决。此

次迁移不仅显著提升了履约模块的性能与稳定性，更让履约系统具备了弹性扩容能力——后续可通过增加 Redis 节点快速应对业务流量增长，为字节国际电商海外业务的持续扩张提供了坚实的技术支撑。

2.3.3 国际电商服务 AI Agent 开发框架搭建（7 月 16 日 —8 月 5 日）

7 月 16 日至 8 月 5 日，我依托字节跳动内部 AI 开发平台的核心能力，主导完成了国际电商服务 AI Agent 开发框架的从零到一搭建与全流程落地。这一项目的启动，源于国际电商业务各核心环节长期存在的效率痛点——随着海外业务版图持续扩张，订单管理、商品运营、物流协同、采购统筹等多个业务域，涌现出大量重复性的数据分析、流程咨询与问题排查需求，传统人工响应模式已难以匹配业务高速发展的节奏。

在传统模式下，这些需求均依赖人工对接响应：业务人员遇到订单履约异常、商品库存预警、物流时效延误等问题时，需线下咨询技术或运营专家，平均响应时间超过 2 小时；而月度销量报表生成、跨境业务合规校验、商家运营数据统计等重复性工作，往往需要专人花费 1-2 天时间整理输出，不仅效率低下，还占用了大量核心人力成本，导致团队难以聚焦高价值的业务创新工作。更关键的是，此前各业务团队曾尝试自行开发 AI 辅助工具，但受限于大模型适配、专业知识库构建、交互逻辑开发等技术门槛，普遍存在开发周期长、功能碎片化、重复造轮子等问题，开发出的工具仅能覆盖单一业务场景，无法形成规模化、标准化的智能服务能力，难以支撑全链路业务的智能化升级。因此，搭建一套统一、低门槛的 AI Agent 开发框架，让各业务域无需关注底层技术实现，就能快速拥有专属 AI 能力，成为提升国际电商全链路运营效率的关键需求。

项目启动后，我首先牵头完成了全面的需求调研与技术选型工作。为确保框架能精准匹配业务痛点，我先后对接了订单管理、商品运营、物流协同、采购统筹、售后服务等 8 个核心业务域的负责人与一线业务人员，通过多轮访谈、需求研讨会，系统梳理出各域的核心诉求：例如商品运营团队需要快速生成区域销量预测报表、订单管理团队需实时诊断异常订单成因、物流协同团队

希望获取跨境运输的时效优化建议等。基于这些需求，我明确了框架需具备的四大核心能力：业务知识问答（如跨境合规政策查询、系统操作指引）、定制化数据报表生成（如销量趋势、履约效率统计）、全流程异常诊断（如订单卡单、支付失败原因排查）、场景化智能决策建议（如库存补货策略、物流渠道选择）。

技术选型阶段，我对比了多种内部 AI 开发平台的技术特性、稳定性与可扩展性，最终确定基于字节内部成熟的 Eino AI 开发平台构建框架——该平台具备完善的大模型调度、向量检索、多智能体协作能力，可直接复用底层技术组件，大幅降低框架开发的技术难度与周期。架构设计上，我采用“核心引擎层-业务适配层-交互层”的三层架构模式，确保框架的灵活性、可扩展性与低门槛接入特性：核心引擎层作为框架的“技术底座”，负责承载大模型调用、知识库向量检索、用户意图识别、指令解析等底层核心能力，通过封装平台 API，实现对多类大模型的统一调度；业务适配层是框架的“桥梁”，提供标准化的接口规范与可视化的提示词模板引擎，支持业务人员无需编写代码，就能快速配置专属业务逻辑；交互层作为“入口”，基于飞书机器人形态实现，让业务人员可通过自然语言直接发起交互，降低使用门槛。

进入开发阶段，我重点攻克了两大核心技术难点。第一个难点是低代码接入能力的实现，这也是框架“降低开发门槛”的核心支撑。我设计并开发了通用的提示词模板引擎，内置了订单、商品、物流等多业务域的基础模板，业务人员只需通过可视化界面，拖拽配置提示词参数、绑定本域的业务接口（如库存查询接口、销量统计接口），即可生成专属的业务 AI Agent，无需关注大模型调用、数据格式转换等底层细节。同时，为解决业务知识精准问答的问题，我开发了知识库管理模块，支持 PDF、Excel、Word、纯文本等多格式文件的批量导入，通过向量数据库将非结构化的业务文档（如跨境电商运营手册、合规政策文件）转化为可检索的向量数据，结合大模型的上下文理解能力，实现“提问-检索-精准回答”的闭环，确保 AI Agent 能精准响应业务人员的知识咨询需求。

第二个难点是大模型调用的性能优化。初期测试发现，大模型响应时间过

长（平均 2-3 秒），且高频重复查询（如每日固定的销量统计）会浪费大量计算资源，影响用户体验。针对这一问题，我引入了多级请求缓存机制：对 1 小时内的高频重复查询请求，直接从缓存中返回结果，缓存命中率最高可达 40%，大幅缩短了响应时间；对复杂的报表生成、多步骤决策类请求，采用异步调用模式，先返回“处理中”提示，完成后通过飞书消息推送结果，避免用户长时间等待导致的交互卡顿。此外，我还设计了多模型路由策略，通过分析请求类型（如简单问答、复杂分析、创意生成），自动匹配性能与效果最优的大模型：简单的业务咨询匹配轻量模型，确保响应速度；复杂的数据分析与决策建议匹配能力更强的大模型，保障结果准确性，实现响应速度与回答质量的平衡。

为进一步提升框架的实用性与易用性，我在交互层做了大量优化：将框架封装为飞书机器人形态，开发了简洁的操作指引与命令菜单，业务人员只需在飞书聊天窗口输入自然语言指令（如“查询北美站近 7 天的订单履约率”“排查订单号 123456 的支付失败原因”），即可触发 AI Agent 的能力。同时，我还开发了配套的权限管理功能，按业务域划分使用权限，确保数据安全；新增操作日志模块，记录所有交互请求与响应结果，方便问题追溯与框架优化；支持报表结果的 Excel 导出，满足业务人员线下分析的需求。

经过 20 天的开发、测试与优化，框架于 8 月 5 日正式上线。上线后，通过内部推广快速覆盖国际电商全业务域，累计服务 20+开发与业务人员，响应各类业务请求 5000+次。实际应用数据显示，该框架将各业务域专属 AI 工具的开发周期从平均 2 周大幅缩短至 1 天，开发成本降低 90%以上；基于专属业务知识库的问答准确率达 92%，异常问题诊断效率提升 60%，定制化报表生成时间从数小时缩短至分钟级。框架的落地不仅解决了各业务域的效率痛点，更推动国际电商业务从“人工驱动”向“智能驱动”转型，为后续业务的规模化扩张提供了高效的智能支撑。

2.3.4 包裹揽收前轨迹功能开发（8 月 6 日 —8 月 22 日）

8 月 6 日至 8 月 22 日，我独立牵头负责包裹揽收前轨迹新增功能的全流程开发工作。该功能的核心目标是补全国际电商物流轨迹的全链路可视化，彻底

解决传统模式下包裹从仓库出库到物流商揽收前的轨迹信息缺失问题，从而提升用户购物体验与物流管理效率。在当时的业务场景中，包裹从商品出库、分拣、打包到最终等待物流商揽收，需经过多个流转环节，且各环节节奏快、节点密集，但这些关键流程的状态数据始终处于“黑盒”状态——用户无法实时知晓订单是否已出库、是否在分拣，只能被动等待揽收信息；物流管理团队也难以监控仓库内部流转效率，一旦出现分拣延误、打包遗漏等问题，无法及时定位根源，导致用户咨询物流进度的工单量居高不下，同时也影响了整体效率。

项目开发初期，我深度参与了需求评审与技术方案设计的全流程。为确保功能精准匹配业务痛点，我先后组织了 3 轮跨团队沟通会，对接产品经理、物流业务负责人及一线运营人员，系统梳理出各角色的核心诉求。最终明确功能需实现三大核心目标：一是实时消费采购方通过 EventBus 消息队列推送的轨迹事件，覆盖商品出库、分拣完成、打包完毕、待揽收 4 个核心节点，确保轨迹数据无遗漏；二是完成轨迹数据的标准化落库存储，并实时同步上报至物流侧核心系统，保障数据在全业务链路的一致性；三是攻克高并发场景下的技术难点——由于同一履约单元的多个轨迹节点可能在短时间内集中推送，极易出现并发写入导致的数据覆盖问题，必须设计可靠的并发控制机制。

基于明确的需求，我主导制定了“消息消费-数据处理-分布式锁控制-轨迹上报”的全链路技术方案，将功能拆解为四个核心模块，确保开发过程有序推进。在开发实施阶段，我首先聚焦消息订阅与解析模块的开发：通过监听 EventBus 指定的消息主题，精准捕获采购方推送的轨迹数据；为保障数据质量，我设计了多层级的标准化校验逻辑，先校验消息格式是否符合预设的 Protobuf 定义，过滤字段缺失、格式错误的非法数据；再通过 Redis 的 Set 结构记录已消费的消息 ID，实现重复消息的快速过滤，避免重复处理导致的数据冗余。随后，我完成了轨迹数据表的结构设计，核心字段涵盖履约单元 ID、轨迹节点编码、操作时间、操作人员 ID、仓库编码等关键信息，同时专门建立了履约单元 ID 与订单号的联合索引——既保证了按订单号查询轨迹的高效性，也支持通过履约单元 ID 追溯全链路状态，为后续数据查询与统计分析奠定基础。

高并发场景下的轨迹数据覆盖问题，是本次开发的核心技术攻坚点。经过技术调研与方案对比，我最终确定采用 Redis 分布式锁机制解决这一问题。具体实现上，以履约单元 ID 作为锁的唯一标识，通过 Redis 的“SET NX EX”命令实现原子性加锁——该命令能确保只有第一个请求能成功创建锁，同时通过 EX 参数设置合理的锁过期时间（结合业务实际设置为 30 秒），有效避免因服务异常崩溃导致的死锁问题。为保障解锁的安全性，我编写了 Lua 脚本实现原子性解锁操作，先校验当前持有锁的线程标识是否与解锁请求一致，再执行删除锁的操作，彻底杜绝误删其他线程持有锁的风险。在轨迹数据写入流程中，我严格遵循“先加锁、再写入、后解锁”的逻辑，确保同一履约单元的多条轨迹消息串行处理，从根本上解决了并发写入的数据覆盖问题。此外，针对消息消费过程中可能出现的网络中断、服务重启等异常场景，我还开发了完善的消息重试机制：将消费失败的消息存入死信队列，通过定时任务按指数退避策略进行重试，同时记录详细的异常日志（包含消息内容、失败原因、重试次数等），便于问题追溯与排查，确保轨迹数据的完整性与可靠性。

功能开发完成后，我牵头开展了多维度、全场景的测试验证工作，确保功能上线后稳定可靠。性能测试方面，我使用 JMeter 模拟每秒 1000+ 的轨迹消息推送场景，持续压测 2 小时，重点监控分布式锁的竞争情况、消息处理吞吐量及接口响应时间，测试结果显示锁竞争成功率达 100%，无数据覆盖问题，消息处理延迟稳定在 50ms 以内；联调测试方面，我联合物流侧团队搭建跨系统测试环境，逐一验证轨迹数据从写入数据库到上报物流系统的全链路同步准确性，确保两端数据实时对齐；异常测试方面，我模拟了消息丢失、网络中断、服务重启、锁超时等多种异常场景，验证重试机制与死锁预防逻辑的有效性，最终所有异常场景均能正常恢复，未出现数据丢失或不一致问题。

8 月 22 日，包裹揽收前轨迹功能顺利完成全量上线。上线后的数据监测显示，该功能成功补全了揽收前的物流轨迹盲区，用户可通过前端订单详情页实时查看商品从出库、分拣、打包到待揽收的全流程状态，物流信息透明度显著提升；物流管理团队借助轨迹数据，能够精准监控仓库内部各流转环节的效

率，及时发现并优化分拣延误、打包瓶颈等问题。业务数据反馈显示，功能上线后用户咨询物流进度的工单量下降 40%，极大减轻了运营团队的工作压力；同时，全链路轨迹数据的完善也为后续物流效率优化、仓库资源调配提供了精准的数据支撑，进一步提升了国际电商物流链路的整体运营效率。

第3章 实习照片

在实习期间，我用相机记录下了许多重要的工作瞬间，这些照片不仅是我实习生活的珍贵回忆，更是我成长与进步的见证。



图 1

办公区走廊（图 1）：2025 年 5 月 23 日，入职第一天的午休时间，我抱着沉甸甸的入职礼包，在办公区走廊拍下了这张照片。左侧电子屏上滚动着 17、18 楼的会议室排期，“17-02 跨境电商技术评审会 14:00”的字样刚好停在屏幕中间，让我对下午的培训多了几分期待；右侧玻璃门后的办公区里，能看到一位同事正对着电脑敲代码，指尖在键盘上的节奏像是在跟系统对话；门口立着的活动海报印着“ByteDance 健康日”的字样，画着两个舒展身体的小人，瞬间让陌生的环境多了些温度。我捏了捏手里印着公司 logo 的笔记本，突然觉得“成为字节技术团队一员”的实感，就藏在这明亮走廊的每一个细节里。



工位日常（图 2）：2025 年 6 月 10 日，完成订单导出接口优化的傍晚，我靠在椅背上拍下了工位区域的画面。我的工位上，装着冰咖啡的透明水壶还凝着水珠，旁边的双屏显示器左边停留在 Excelize 库的 API 文档页面，右边是刚跑完的压力测试报告——“百万级数据导出无内存溢出”的绿色提示还亮着；邻座同事的椅子上搭着件浅蓝色的防晒外套，椅轮还卡在桌腿旁，像是主人刚临时起身去沟通问题；更远处的椅子上，一个毛绒绒的橙色坐垫歪歪地铺着，是团队里的前辈之前分享的“久坐神器”。桌上的数据线缠成温柔的弧度，键盘上还沾着半块没吃完的薄荷糖，这些细碎的“杂乱”，裹着我们一起熬到凌晨调索引、围着屏幕改 bug 的热乎劲儿，成了“攻坚时刻”最鲜活的注脚。



休闲区入口（图3）：2025年7月20日，履约存储库迁移项目通过全量测试的那天下午，我抱着笔记本想找个地方整理复盘文档，走到了休闲区入口。玻璃门两侧印着“ByteDance 字节跳动”的灰色标识，门后的空间里，浅白色的桌椅错落摆着，几张桌子上还留着没收拾的咖啡杯和笔记本，像是刚结束一场小型讨论；远处的落地窗外，城市的天际线裹在浅蓝的天光里，风把窗帘吹得

轻轻晃；入口旁的墙面上，箭头标识指着“10-12 会议室”和“01-09 办公区”，绿色的紧急出口灯在角落亮着柔和的光。我推开门时，刚好闻到里面飘来的茶包香气，突然觉得那些熬到眼睛发涩的夜晚、对着日志反复调试的焦虑，都能被这样松弛的空间轻轻接住——原来高效的开发节奏里，也藏着这样温柔的“缓冲地带”。

短信

9月5日 19:09 2

【字节跳动】亲爱的杨梓枫：

感谢你曾经作为一名 ByteDancer 付出的努力。在字节跳动，我们曾携手同行，践行“激发创造，丰富生活”的使命。在这段旅程中，我们共同为用户、创作者和合作伙伴创造了更大的价值。

祝愿你未来一切顺利！

字节跳动人力与管理

Thank you for your hard work and dedication to the company during your time with us. Thanks to you, we are one step closer to achieving our mission to "Inspire Creativity, Enrich Life," creating value for users, creators and collaborators along the way.

All the best to you!

ByteDance Human Resources and Management

相关资讯

离职告别短信（图4）：2025年9月5日19:09，我收到了这条来自字节跳动人力与管理的短信，顺手截下了屏幕。看着短信里“曾经作为一名

ByteDancer”的字样，指尖在屏幕上顿了好一会儿——想起5月23日抱着入职礼包走进走廊的紧张，想起6月10日盯着订单导出接口测试报告的雀跃，想起7月20日在休闲区闻着茶包香气整理复盘的松弛，这些画面突然都涌了上来。短信里“激发创造，丰富生活”的使命，是我敲每一行代码、改每一个bug时都记着的方向；那句“共同创造了更大的价值”，让这三个多月里熬的夜、问的每一个“为什么”都有了沉甸甸的温度。我把这条短信存在了相册最下面的文件夹里，就像把这段带着代码温度的旅程，好好收进了青春的行囊里。

3.1 实习证明

实习结束后，珠海聚横易行网络科技有限公司为我出具了正式的实习证明。证明中明确确认我于2025年5月23日至9月5日在字节国际电商部门担任后端开发实习生（Golang）。在实习期间，我积极参与了多个核心接口的开发与优化工作，从需求分析、设计、编码实现到测试、上线，每一个环节都认真负责，努力完成各项任务。我严格遵守公司的规章制度，积极与团队成员沟通协作，不断提升自己的技术能力和业务水平，表现良好，得到了公司领导和同事的一致认可。实习证明扫描件见附录1，它将成为我未来职业发展道路上的一份重要见证，激励我不断前进。

第4章 实习收获与总结

4.1 实习收获

4.1.1 技术能力提升

编程语言与框架：通过实习期间的大量实践，我对 Golang 编程语言的掌握达到了一个新的高度。不仅熟练运用其核心特性进行后端接口开发，还深入理解了 Golang 的内存管理机制，能够避免常见的内存泄漏和资源浪费问题。在并发编程方面，我熟练掌握了 Goroutine 和 Channel 的使用技巧，能够根据业务需求合理地设计并发任务，实现高效的异步处理。例如，在订单导出功能中，通过使用 Goroutine 并发读取数据库数据和生成 Excel 文件，将原本需要数分钟的导出时间缩短至几十秒，大大提高了用户体验。在微服务架构设计中，我深入学习了 gRPC 微服务通信机制，能够独立完成 gRPC 服务的定义、实现和调用。掌握了如何使用 protobuf 定义接口和消息结构，以及如何通过代码生成工具生成高效的 Golang 代码。通过实践，我深刻体会到 gRPC 在提高服务间通信效率和性能方面的优势，为构建高性能的分布式系统奠定了坚实的基础。

数据库与中间件：在数据库方面，我精通了 MySQL 数据库的高级应用，能够熟练编写复杂的 SQL 语句，实现数据的高效查询、更新和管理。通过深入学习数据库索引优化技术，我能够根据业务查询需求，为数据库表创建合适的索引，大大提高了查询性能。在处理海量订单数据时，通过优化索引，将查询时间从秒级缩短到了毫秒级。同时，我还掌握了慢查询排查的方法，能够通过分析慢查询日志，找出性能瓶颈并进行针对性的优化。在中间件应用方面，我熟练掌握了 Redis 缓存技术的各种应用场景，能够根据业务需求合理设置缓存策略。例如，对于热门商品详情页面，设置较长的缓存过期时间，减少数据库查询压力；对于订单相关数据，采用读写分离的缓存策略，确保数据的一致性和实时性。在处理高并发场景时，通过使用 Redis 分布式锁，有效地解决了资源竞争问题，保证了数据的完整性。此外，我还深入学习了 Kafka 消息队列的原理和应用，能够利用 Kafka 实现系统间的异步通信和解耦。在订单支付流程

中，通过 Kafka 将支付结果消息发送给相关服务进行后续处理，大大提高了系统的响应速度和稳定性。

工具与流程：在开发工具方面，我熟练掌握了 Goland 的高级功能，如代码调试、性能分析、代码重构等，能够高效地进行代码开发和优化。通过使用 Goland 的代码分析工具，能够及时发现代码中的潜在问题和风险，并进行修复，提高了代码的质量和可靠性。在版本控制方面，我精通 Git 的各种操作，包括分支管理、代码合并、冲突解决等，能够熟练地进行团队协作开发。在项目开发过程中，通过合理使用 Git 分支，实现了不同功能模块的并行开发，提高了开发效率。同时，我还学会了使用 Jira 进行项目管理，能够熟练创建任务、分配任务、跟踪进度和管理问题，确保项目的顺利进行。在敏捷开发流程方面，我深入理解了敏捷开发的理念和方法，能够熟练参与迭代开发、每日站会、代码评审等活动。通过每日站会，及时沟通项目进展和问题，确保团队成员之间的信息共享和协作。在代码评审中，能够从代码质量、可读性、可维护性等多个角度对代码进行审查，提出合理的改进建议，提高了团队整体的代码水平。此外，我还学习了 Docker 容器化部署的高级技术，能够熟练构建 Docker 镜像、配置容器网络和存储，实现应用的快速部署和迁移。通过使用 Docker，实现了开发、测试和生产环境的一致性，提高了应用的可靠性和可维护性。

问题排查与优化：在实习期间，我掌握了一系列线上问题排查的工具和方法，能够快速定位和解决系统中出现的各种问题。通过使用监控工具（如 Prometheus、Grafana），实时监控系统的性能指标，包括 CPU 使用率、内存使用率、接口响应时间等，及时发现系统的异常情况。在发现接口响应时间过长的问题时，通过监控工具定位到是某个数据库查询语句导致的性能瓶颈。然后，通过分析日志工具（如 ELK），深入分析系统日志，找出问题的根本原因。在这个案例中，通过查看慢查询日志，发现该查询语句没有使用合适的索引，导致查询时间过长。最后，通过链路追踪工具（如 Jaeger），跟踪请求在系统中的执行路径，进一步确认问题的所在。在定位到问题后，我提出了相应的

优化方案，如为该查询语句添加合适的索引、优化查询逻辑等，成功解决了接口响应时间过长的问题，提升了系统的性能和稳定性。在缓存优化方面，我深入学习了缓存穿透、缓存击穿、缓存雪崩等问题的解决方案。通过使用布隆过滤器，有效地解决了缓存穿透问题，防止了大量非法请求穿透缓存直接访问数据库。在处理缓存击穿问题时，采用互斥锁的方式，保证在高并发情况下只有一个请求能够访问数据库，避免了缓存击穿导致的数据库压力过大。对于缓存雪崩问题，通过设置不同的缓存过期时间，避免了大量缓存同时过期的情况，保证了系统的稳定性。

4.1.2 业务理解加深

通过参与字节国际电商后端系统的开发，我对国际电商业务的核心逻辑和流程有了深入的理解。在订单管理模块，我全面掌握了订单从创建到完成的全生命周期管理，包括订单的创建、支付、履约、退款、统计等环节。在订单创建过程中，需要考虑用户信息的验证、商品库存的检查、促销活动的计算等多个因素，确保订单的合法性和准确性。在订单支付环节，对接第三方支付平台，实现了安全、可靠的支付流程，并处理了支付结果回调、异常处理等复杂业务逻辑。在履约过程中，与物流系统对接，实现了订单的发货、运输跟踪等功能。在退款环节，需要根据不同的退款原因和规则，进行退款金额的计算、库存的回滚等操作。在商品库存管理中，我深刻理解了库存扣减的业务逻辑和重要性，通过分布式锁技术解决了高并发场景下库存超卖的问题。在库存扣减时，需要先检查库存是否充足，然后使用分布式锁确保同一时间只有一个线程能够进行库存扣减操作，保证了库存数据的一致性和准确性。在支付履约方面，我了解了支付的各种方式和流程，以及如何与第三方支付机构进行对接和结算。在物流配送模块，我熟悉了与第三方物流 API 对接的流程，实现了物流信息的实时查询和跟踪，为用户提供了良好的购物体验。同时，我还深入了解了跨境电商的政策法规和行业特点，包括不同国家和地区的税收政策、海关监管要求、物流配送特点等。这些知识使我能够更好地理解业务需求，为系统的开发和优化提供了有力的支持。例如，在设计订单履约流程时，需要考虑不同国家的物流规则和运输时间，合理安排发货和配送计划，以提高用户满意度。

4.1.3 团队协作能力提升

在实习过程中，我积极参与团队协作，通过各种团队活动，如需求评审、代码评审、技术分享等，与团队成员建立了良好的合作关系，提升了沟通协调能力和团队合作能力。在需求评审中，我认真倾听产品经理的需求讲解，积极提出自己的疑问和建议，与团队成员共同探讨技术实现方案。在这个过程中，我学会了从不同角度思考问题，理解业务需求的本质，确保技术方案能够满足业务需求。例如，在讨论订单列表查询接口的需求时，我提出了优化查询性能的建议，通过添加合适的索引和优化查询逻辑，提高了接口的响应速度，得到了团队成员的认可。在代码评审中，我认真审查团队成员的代码，从代码质量、可读性、可维护性等多个角度提出意见和建议。同时，我也虚心接受他人对我的代码的评审意见，不断改进自己的编程习惯和代码质量。通过代码评审，不仅提高了代码的质量，还促进了团队成员之间的技术交流和學習。在技术分享中，我积极分享自己在技术学习和实践中的经验和成果，同时也认真学习团队成员分享的技术知识和项目经验。通过技术分享，拓宽了自己的技术视野，学习了新的技术和方法，提升了自己的技术水平。例如，在一次技术分享中，我了解到了一种新的缓存优化方案，通过在项目中应用该方案，有效地提升了系统的性能。在敏捷开发过程中，我学会了合理规划任务，主动与团队成员协作，共同解决开发中的问题。在迭代开发中，根据任务的优先级和难度，合理分配时间和资源，确保任务能够按时完成。在遇到问题时，主动与团队成员沟通，共同分析问题的原因，寻找解决方案。例如，在订单导出接口优化过程中，与前端开发人员紧密配合，共同解决了数据格式不一致、接口响应时间过长等问题，确保了功能的顺利上线。

4.1.4 职业素养养成

通过本次实习，我培养了严谨的工作态度，注重代码质量和文档规范。在代码编写过程中，严格遵循团队的编码规范，注重代码的可读性、可维护性和可扩展性。在编写订单状态变更接口代码时，对每一个状态变更的逻辑都进行了详细的注释，确保代码的可读性和可维护性。同时，养成了编写单元测试的良好习惯，通过编写单元测试用例，对代码的功能进行全面的测试，确保代码

的正确性和稳定性。在完成订单导出接口开发后，编写了详细的单元测试用例，覆盖了正常导出、异常参数、大数据量导出等多种场景，通过测试及时发现并解决了一些潜在的问题。在项目开发过程中，学会了合理安排时间，提高工作效率。在面对多个任务和紧张的工期时，制定详细的工作计划，按照任务的优先级和时间节点进行有序的工作。在项目迭代高峰期，通过合理安排时间，成功完成了多个核心接口的开发和优化任务，确保了项目的按时交付。同时，增强了责任意识，了解了线上系统的重要性。在处理线上问题时，认真对待每一个细节，确保问题能够得到及时、有效的解决。在一次线上订单查询接口出现异常时，迅速响应，通过分析日志和监控数据，及时定位到问题的原因，并采取了有效的措施进行修复，确保了系统的稳定运行。

4.2 问题与反思

技术深度不足：尽管在实习期间掌握了多种技术的基本使用方法，但在技术深度上仍存在欠缺。对于分布式事务，如 Seata 框架的原理和应用，理解还不够深入，在实际项目中遇到复杂的分布式事务场景时，难以快速找到最佳的解决方案。在微服务治理方面，对于服务熔断、服务降级等机制的理解和应用还不够熟练，缺乏在大规模高并发系统中进行微服务治理的经验。在高并发系统设计中，对于如何优化系统架构、提高系统的吞吐量和响应速度，还需要进一步学习和实践。为了提升技术深度，我计划在未来通过阅读专业的技术书籍，如《分布式事务原理与实战》《微服务架构设计模式》等，深入学习相关技术的原理和应用。同时，积极参与开源项目，通过实践积累经验，提升自己在复杂技术领域的能力。

业务细节理解偏差：在实习过程中，部分业务场景由于对需求理解不全面，导致开发返工。在处理退款流程时，未充分考虑跨境支付手续费的计算，导致退款金额计算错误，需要重新调整代码逻辑。在商品库存管理中，对于库存预警机制的理解不够深入，导致库存预警功能无法满足业务需求。为了避免类似问题的发生，我将加强与产品经理和业务人员的沟通，在需求分析阶段，详细了解业务背景和需求细节，确保对业务需求的准确理解。同时，在开发过

程中，及时与业务人员进行确认，避免因理解偏差导致的开发返工。

沟通表达能力待提升：在需求汇报和技术分享中，我存在表达不清晰、重点不突出的问题。在一次需求汇报中，由于没有清晰地阐述需求的背景和目标，导致团队成员对需求的理解出现偏差，影响了项目的进度。在技术分享中，由于语言组织能力不足，没有将技术要点和实践经验清晰地传达给团队成员，降低了分享的效果。为了提高沟通表达能力，我将通过模拟练习，如进行需求汇报和技术分享的模拟演讲，不断改进自己的表达方式和沟通技巧。同时，学习有效的沟通方法和技巧，如结构化表达、倾听技巧等，提高自己的沟通能力。

抗压能力需加强：在项目高峰期，面对多任务处理时，我偶有焦虑情绪，影响了工作效率。在一次项目迭代中，同时负责多个接口的开发和优化任务，由于时间紧迫，任务繁重，导致自己出现了焦虑情绪，影响了代码的编写质量和工作进度。为了提升抗压能力，我将优化时间管理，采用优先级排序法，对任务进行合理的优先级划分，确保重要任务能够按时完成。同时，通过学习放松技巧，如冥想、运动等，缓解工作压力，保持良好的心态。

4.3 未来展望

深化技术学习：未来，我将聚焦分布式系统、微服务治理、高并发架构设计等领域，深入学习前沿技术和最佳实践。计划阅读《分布式系统原理与范型》《高并发系统设计》等经典书籍，系统学习分布式系统的理论知识和实践经验。积极参与开源项目，如 Kubernetes、Dubbo 等，通过贡献代码和参与社区讨论，深入了解微服务治理和高并发架构设计的实际应用。同时，关注行业技术动态，定期参加技术研讨会和线上技术交流活动，学习最新的技术发展趋势和应用案例。通过撰写技术博客，分享自己的学习心得和实践经验，巩固所学知识，与同行进行交流和互动。

加强业务积累：我会持续关注跨境电商行业的动态，深入学习国际物流、支付结算、海关监管等业务知识。订阅行业权威媒体和资讯平台，及时了解行业政策法规的变化和市场趋势。主动参与业务需求分析和项目规划，与业务人

员紧密合作，深入了解业务需求的本质，提供更贴合业务需求的技术解决方案。在实际项目中，不断积累业务经验，提升自己对业务的理解和把握能力，实现从技术执行者到业务赋能者的转变。

提升综合素质：积极参与团队管理和技术分享活动，锻炼自己的沟通协调能力和团队管理能力。在团队管理中，学习项目管理知识，如项目计划制定、任务分配、进度跟踪等，提高团队协作效率。在技术分享中，不断提升自己的表达能力和技术水平，将自己的技术经验和知识有效地传递给团队成员。注重培养创新思维和领导力，在工作中积极提出新的想法和建议，推动技术创新和业务发展。同时，制定个人职业规划，设定短期和长期目标，如在一年内掌握分布式事务的核心技术，在三年内成为技术骨干，逐步成长为技术与业务兼备的复合型人才。

明确职业方向：以 Golang 后端开发为核心方向，深耕电商领域。在未来 3-5 年内，积累丰富的电商行业经验，成为一名资深后端工程师。能够独立设计和优化高并发电商系统，解决复杂的技术难题，为企业的技术创新和业务发展贡献力量。在技术方面，不断提升自己在 Golang 语言、分布式系统、微服务架构等领域的技术水平，掌握最新的技术工具和框架。在业务方面，深入了解电商行业的业务模式和发展趋势，能够根据业务需求提供高效的技术解决方案。通过持续学习和实践，不断提升自己的竞争力，实现自己的职业目标。

参考文献

- [1] 许式伟. Go 语言编程 [M]. 北京: 机械工业出版社, 2023.
- [2] 马丁·福勒. 微服务架构设计模式 [M]. 北京: 人民邮电出版社, 2022.
- [3] 周志明. 深入理解 Java 虚拟机 (第 4 版) [M]. 北京: 机械工业出版社, 2021.
- [4] 阿里巴巴集团. 阿里巴巴 Java 开发手册 (泰山版) [M]. 北京: 电子工业出版社, 2020.
- [5] 字节跳动技术团队. 字节跳动微服务实践 [J]. 计算机工程与应用, 2024, 60 (12): 1-8.
- [6] Redis 官方文档. <https://redis.io/documentation>
- [7] Go 官方文档. <https://golang.org/doc/>
- [8] 李智慧. 大型网站技术架构: 核心原理与案例分析 [M]. 北京: 机械工业出版社, 2019. 该书系统阐述了大型网站在架构设计、性能优化、高并发处理等方面的核心原理, 并结合众多实际案例进行分析, 对理解电商系统的架构设计和技术选型有很大帮助, 尤其在处理海量订单数据和高并发访问时, 其中的缓存设计、数据库优化等内容为实际工作提供了理论依据。
- [9] 程杰. 大话设计模式 [M]. 北京: 清华大学出版社, 2007. 通过通俗易懂的语言和生动的案例, 介绍了常见的设计模式及其在软件开发中的应用。在开发电商系统的过程中, 设计模式能够帮助优化代码结构, 提高代码的可维护性和可扩展性, 例如在订单状态变更接口的开发中, 运用状态模式可以使代码结构更加清晰, 易于维护。
- [10] 郑晔. 软件设计之美 [M]. 北京: 电子工业出版社, 2020. 从软件设计的多个维度, 如架构设计、代码质量、团队协作等, 探讨了如何打造高质量的软件。在实习期间, 对于理解软件设计的原则和方法, 以及如何在团队协作中遵循这些原则, 提升软件的整体质量有很大的启发。
- [11] 王健宗, 汤银才. 机器学习: 软件工程方法与实现 [M]. 北京: 机械工业出版社, 2020. 介绍了机器学习在软件工程中的应用, 以及如何将机器学习算法与实际业务相结合。虽然本次实习主要聚焦于后端开发, 但在电商业务中, 机器学习技术在用户行为分析、推荐系统等方面有广泛应用, 了解相关知识有助于拓展技术视野, 为未来可能涉及的业务场景做好准备。
- [12] 何海涛. 剑指 Offer: 名企面试官精讲典型编程题 (第 2 版) [M]. 北京: 电子工业出版社, 2017. 书中包含了大量经典的编程面试题及详细解答, 对于提升编程能力和算法思维有很大帮助。在实习过程中, 算法思维在解决一些复杂业务问题和优化代码性能时发挥了重要作用, 如在订单统计接口的开发中, 运用合适的算法提高了大数据量下的查询效

率。

附录

附录 1：实习证明扫描件

2/2

实习证明 - Internship Certificate

兹证明杨梓枫，有效证件号码：450422200310112156，在本公司产品研发和工程架构部门实习，在职期间为2025-05-23至2025-09-05。

This document certifies that Zifeng Yang, valid ID No. 450422200310112156 has worked as an intern in the Product RD and Infrastructure Department of our company from 2025-05-23 to 2025-09-05.

特别提示：实习生在离职后应继续履行对我司的保密义务，不得向第三方泄露我司及/或关联公司的任何保密信息。

This is a reminder that after leaving the Company, the intern shall continue to fulfill the obligation of confidentiality to our company and shall not disclose any confidential information of our company and/or its affiliated companies to any third party.

珠海聚横易行网络科技有限公司
Zhuhai Juheng Yixing Network Technology Co., Ltd.

日期/Date: 2025-09-05

人事专用章

